Level 2 — Data Types and Type Systems

Jaymin C. Amin

School of Science and Technology, United States International University – Africa

APT1030A: Fundamentals of Programming Languages

Dr. Lawrence Nderu

March 22, 2026

↓ Next Page ↓

Level 2 — Data Types and Type Systems

Context: SACCO Member Financial Record

Concepts

- Static vs dynamic typing
- Primitive vs composite types
- Type coercion

Implementation Task

Create a program that stores:

- Member Name
- Member ID
- Monthly Contribution
- Total Savings (calculated)

Requirements:

1. Implement in:
    - Python
    - Java or C++
2. Allow user to input contribution for 6 months.
3. Compute total savings.

Critical Test

Attempt to:

- Assign a string value to contribution.
- Observe behavior in:
    - Python
    - Java/C++

Analytical Question

Explain:

1. Why static typing may be safer for financial systems.

    Static typing enforces strict data rules at the compilation stage. This approach protects sensitive software applications like a SACCO ledger. Languages like Java or C++ require developers to define exact data formats early in the process. The compiler stops the entire program from building if an engineer tries to insert a text string into a numeric calculation field. Early validation stops critical errors during live financial transactions. Handling millions of shillings requires strict exactness to prevent inaccurate user balances and maintain customer trust.

↓ Next Page ↓

2. Where dynamic typing may be advantageous in Kenyan startup environments.

   Dynamic typing helps technology startups build software products fast. Python and JavaScript let your team build Minimum Viable Products without writing rigid boilerplate code. Your startup gains the flexibility to integrate external platforms like MPesa or KRA services rapidly. Business requirements change often. Developers easily adjust underlying data structures to support new member attributes or alter the entire financial model. Strict languages demand heavy code refactoring to achieve similar architectural changes.